Search in course

Search

English

B

1. CS 498: Cloud Computing Applications
2. Week 16
3. MP12 (V2): Two-Tier Microservice Architecture for ML Inference (Optional - Beta Testing)

PreviousNext

Practice Programming Assignment: MP12 (V2): Two-Tier Microservice Architecture for ML Inference (Optional - Beta Testing)

You have not submitted.

Deadline

Pass this assignment by May 11, 11:59 PM CDT

InstructionsMy submissionsDiscussions

1. Overview

In this milestone project, you will deploy a cloud-based machine learning service that supports both a **free** and a **premium** tier. Each tier serves image classification tasks using different neural network models. You will use **Amazon EKS**, **Docker**, and **Kubernetes** to build, deploy, and manage your infrastructure, ensuring that resource usage is controlled and services are isolated.

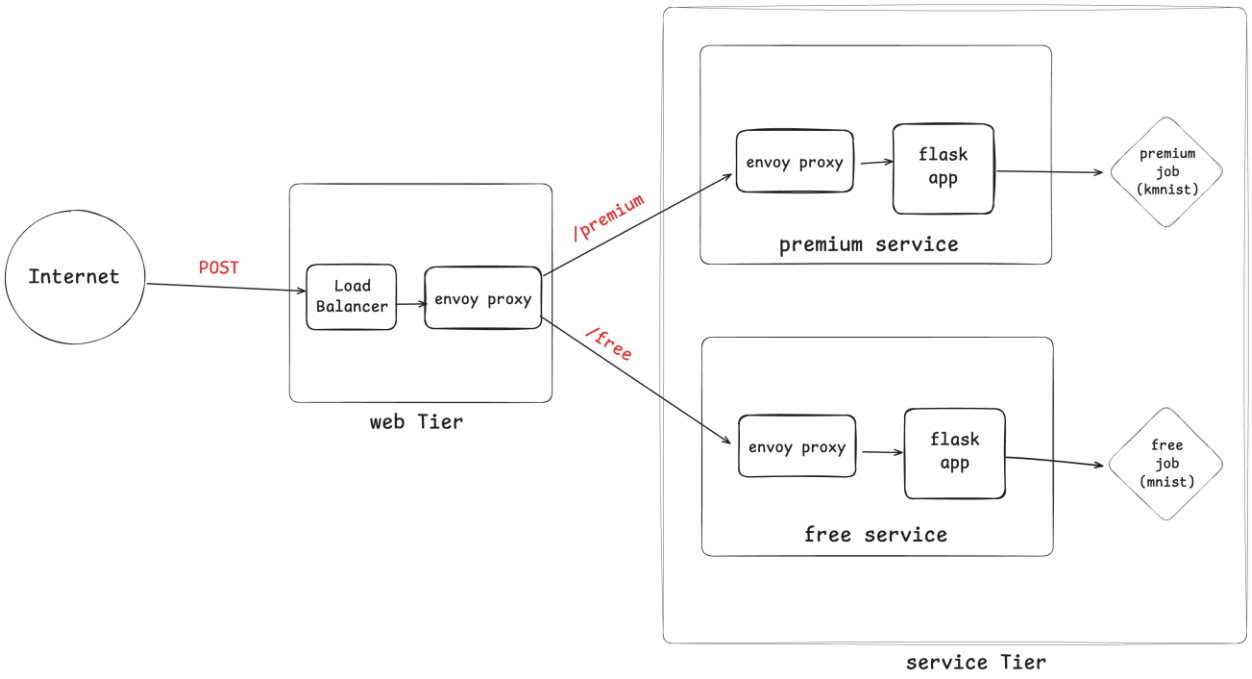
The architecture follows a **two-tier micro-service design**:

- A **Web Tier** (Envoy proxy + Flask app) receives and routes requests.
- A **Service Tier** handling corresponding services:

- A *Free service* serves inference jobs (*/free*) using a simple feed-forward network.
- A *Premium service* serves inference jobs (*/premium*) using a convolutional neural network.

This separation helps manage workloads, enforce resource quotas, and simulate a scalable SaaS offering.

Two Tier Architecture Design:



2. Requirements

NOTE: Region: Use region ***us-east-1*** for your deployment.

21. **Recommended Setup**: It is highly recommended to implement this assignment entirely on an EC2 instance. By using an EC2 instance, you can create Docker images, set up your Kubernetes cluster, and perform all cluster operations from the same instance. This simplifies permissions since both the EC2 instance and the EKS cluster will be in the same AWS account.

2.2. **AWS Account**: AWS Educate accounts will not work as they cannot create the network resources (EKS) required for this assignment.

2.3 **Starter Code**: Kindly use the starter-code to begin your implementation.

3. Process

3.1 Containerization of Service Tier

3.1.1 Create Job Images

Dockerfile.premium-tier-job* and *Dockerfile.free-tier-job

You will create Dockerfiles for the **free-service job** and **premium-service job** containers. These containers will run the classification tasks using the provided *classify.py* script. The models (feed-forward (*ff*) or convolutional neural networks (*cnn*)) and datasets (*MNIST* or *KMNIST*) shall be specified using environment variables.

Environment Variables:

- *DATASET*: Specifies the dataset (*mnist* or *kmnist*).
- *TYPE*: Specifies the model type (*ff* for feed-forward or *cnn* for convolutional neural network).
- Free-service configuration: ff, mnist
- Premium-service configuration: cnn, kmnist

Steps in the Dockerfile:

1. Run the provided *data_preload.py* script to download the training data.
2. Run the *train.py* script to train the models for each dataset.

Expected Output:

When you run the container, you should see output similar to the following:

1

2

3

4

5

6

7

8

9

10

11

dataset: mnist

Downloading http://yann.lecun.com/exdb/mnist/train-images-idx3-ubyte.gz to ./data/MNIST/raw/train-images-idx3-ubyte.gz

100.1%Extracting ./data/MNIST/raw/train-images-idx3-ubyte.gz to ./data/MNIST/raw

...

Processing...

Done!

Epoch [1/5], Step [100/600], Loss: 0.2846

Epoch [1/5], Step [200/600], Loss: 0.3859

Epoch [1/5], Step [300/600], Loss: 0.1448

...

Accuracy of the network on the 10K test images: 98 %

3.1.2 Create Flask App Docker Images

Dockerfile.free-tier-app

The Free-Tier Flask app handles /free requests and creates Kubernetes jobs for free-tier ML tasks. Implement the following files, copy them into the image, and run the flask app:

- *flask_app.py* (TO DO)
- *free-tier-job.yaml* (TO DO)
- *requirements_flaskapp.txt* (provided, DO NOT MODIFY)

Dockerfile.premium-tier-app

The Premium-Tier Flask app handles /premium requests and creates Kubernetes jobs for premium-tier ML tasks. Implement the following files, copy them into the image, and run the flask app:

- *flask_app.py* (TO DO)
- *premium-tier-job.yaml* (TO DO)

- *requirements_flaskapp.txt* (provided, DO NOT MODIFY)

3.1.3 Push Docker Images to Repository

Now that the container image is ready, you need to push the image to a repository/registry. You can either use AWS ECR or Dockerhub *(recommended for ease-of-use)* for this purpose. The worker nodes in the kubernetes cluster will pull the image from the registry and use them for the job.

Dockerhub reference

AWS ECR documentation

3.2 Containerization & Setup of Web Tier

In this step, your objective is to design, build, and deploy the w**eb tier**, which serves as the entry point for your two-tier ML inference system. This layer functions as a **reverse proxy and traffic router**, using **Envoy Proxy** to manage and direct HTTP requests from clients to the appropriate backend Flask services (/free or /premium).

- Your task is to containerize the web tier, configure its routing logic, and deploy it to Kubernetes as a public-facing LoadBalancer service.
- You will build a custom Docker image (*Dockerfile.web-tier*) based on the official Envoy image. This image must include your routing configuration file (*web_tier_envoy.yaml*), which defines how **POST** requests to ***/free*** and ***/premium*** are forwarded to the correct services running within the cluster. The Web Tier must expose a listening port (e.g., 8080), run Envoy with the custom configuration, and route traffic efficiently and accurately based on request endpoints.
- Once containerized, you will define a Kubernetes deployment and service (*web_tier_deployment.yaml*) to run this Web Tier. The service must be of type LoadBalancer so it exposes an external IP address that the autograder and users can access.

3.3 Kubernetes Infrastructure Setup

1. With your components ready, your next objective is to set up a fully functional Kubernetes infrastructure on AWS using Amazon EKS. Prepare host EC2 machine through which you will create and manage your EKS cluster.

You need to install aws cli, kubectl and eksctl in order to create and manage the EKS cluster.

Note 3.3.1: For your EC2 instance to have permissions to create an EKS cluster, you need to either set up the credentials using "aws configure" command (you need to give access key for your account) or use an IAM role for the instance will all the required policies to create a cluster.

2. You are required to provision the cluster using the eksctl tool and a provided *clusterConfig.yaml* configuration file. Use command: `*eksctl create cluster -f clusterConfig.yaml*`
3. **Namespaces**: Once the cluster is live, you must create **three namespaces** to logically separate your services: one for the free tier (*free-service*), one for the premium tier (*premium-service*), and one for the web tier (*web-tier*) **Note 3.3.2:** Premium service pods should execute under the "*premium-service*" namespace and the free ones under the "*free-service*" namespace.
4. **FreeResourceQuota**: To prevent resource starvation, you'll configure the free-service namespace with a **strict resource quota**: no more than 2 CPUs may be used in total, and only 2 free-tier pods should run at any time.
5. **Envoy**: Additionally, you need to prepare and apply Envoy proxy configurations for each tier by creating Kubernetes ConfigMaps from YAML files. This ensures proper routing and service isolation.
6. **Deploy microservices**: Deploy the Flask apps and corresponding services for the free, premium, and web tiers.

A successful setup will establish a multi-tier microservice foundation that meets all namespace and resource isolation requirements.

4. Testing

- Once you have deployed all tiers and exposed the web-tier service using a LoadBalancer, test the setup by sending HTTP POST requests directly to the service IP.

1

2

3

4

5

6

7

8

```

# TEST FREE SERVICE

curl -X POST http://<LoadBalancer-IP>/free -H "Content-Type: application/json" -
d '{"dataset": "mnist"}'


# TEST PREMIUM SERVICE

curl -X POST http://<LoadBalancer-IP>/premium -H "Content-Type: application/json" -
d '{"dataset": "kmnist"}'

```


Some useful commands to sanity check your implementation

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

```
# ============================
# 🛠️ DEBUGGING AND INSPECTION
# ============================


# retrieve the LoadBalancer IP.
kubectl get svc web-tier-service -n web-tier
```

```
# View jobs created in free-service namespace
kubectl get jobs -n free-service

# View jobs created in premium-service namespace
kubectl get jobs -n premium-service

# View logs from Flask app container
kubectl logs <pod-name> -c python-service -n <namespace>

# View logs from Envoy container
kubectl logs <pod-name> -c envoy -n <namespace>

# List all pods in all namespaces
kubectl get pods -A

# List all services in all namespaces
kubectl get services -A

# Describe a specific pod in free-service (see events, status, etc.)
kubectl describe pod <pod-name> -n free-service

# Delete all jobs in free-service namespace
kubectl delete jobs --all -n free-service

# Delete all jobs in premium-service namespace
```

kubectl delete jobs --all -n premium-service

5. Submission

- After completing the deployment and configuration of your two-tier ML inference service, you will validate your solution by running the *submit.py* . Ensure *grader_interface.py* is running on your EC2.
- **Note**: *grader_interface.py* script should be executed from your EC2 instance, where your Kubernetes cluster access is already set up.
- You need the following for your submission:
    - **Load balancer IP**: Can use the following command to get the EXTERNAL IP of your LoadBalancer. ` *kubectl get svc web-tier-service -n web-tier*`
    - **EC2 IP**: EC2 Instance's IP, which is running the *grader_interface.py*

Autograder Tests the following:

- **Test 1:** Sends a POST request to the */free* endpoint via the Load Balancer.
- **Test 2:** Sends a GET request to */free/resource-quota* to inspect Kubernetes quota configuration.
- **Test 3:** Submits a job using the */free* endpoint and waits for it to complete.
- **Test 4:** Submits a job using the */premium* endpoint and waits for it to complete. Then, sends a GET request to */premium/logs* to fetch execution logs.

6. Resource Cleanup

To clean up all resources after submission, begin by deleting the EKS cluster using the command `eksctl delete cluster --name <cluster-name>`. If needed, also manually remove all associated resources:

1. Delete the EKS cluster: `eksctl delete cluster --name <cluster-name> `
2. In the EC2 Console, manually remove all Auto Scaling Groups.
3. Terminate any running EC2 instances.
4. Delete CloudFormation/stacks
5. Release Elastic IPs under the EC2, Elastic IPs section.
6. Delete any EBS volumes from the EC2, Volumes tab.

7. Remove Load Balancers through EC2, Load Balancing, Load Balancers.
8. Delete the VPC after ensuring no dependent resources remain.
9. In the IAM Console, delete any custom IAM roles, policies, and security groups associated with the project.

How to submit

Copy the token below and run the submission script included in the assignment download. When prompted, use your email address **fridkis2@illinois.edu**.

sT97FhYnb0GupuaN

Generate new token

Your submission token is unique to you and should not be shared with anyone. You may submit as many times as you like.

Like

Dislike

Report an issue